

Comparative Analysis of ChatGPT, DeepSeek, and Gemini for Automated Code Generation

Khuram Shahzad

*School of Electrical Engineering & Computer Science
National University of Sciences and Technology (NUST)*

Islamabad, Pakistan

kshahzad.dphd18seecs@seecs.edu.pk

Sohail Iqbal

*School of Electrical Engineering & Computer Science
National University of Sciences and Technology (NUST)*

Islamabad, Pakistan

sohail.iqbal@seecs.nust.edu.pk

Abstract—Code generation using artificial intelligence (AI) has revolutionized software development, providing automated coding solutions. This study conducts a systematic comparative analysis of three leading large language models (LLMs) such as ChatGPT (O1), DeepSeek (R1) and Gemini (2.0 Flash thinking), for Python code generation, evaluating their performance in correctness, code quality, and computational efficiency. Using a curated dataset of Codeforces programming problems that span various difficulty levels (800-2000 complexity), the research employs a rigorous evaluation framework that integrates online judge validation, static code analysis, and runtime profiling. The experimental results reveal that DeepSeek achieves comparatively higher correctness by consistently producing accepted solutions in fewer attempts, although its reasoning time increases with problem complexity. Gemini, on the other hand, is remarkably fast, delivering results in a fraction of the time, but its correctness deteriorates on more complex tasks. ChatGPT offers balanced performance with intermediate correctness and efficiency; however, it sometimes exhibits lower code quality. Overall, our findings underscore the inherent trade-offs between efficiency, accuracy, and quality in AI-generated code. The study provides actionable insights for developers, emphasizing the need to align model selection with project requirements, and contributes a replicable framework for future evaluations of AI code generation tools.

Keywords—Software Development, Artificial Intelligence, ChatGPT, DeepSeek, Gemini, Large Language Model

I. INTRODUCTION

The integration of AI into software development has ushered in a transformative era, redefining how code is conceived, written, and optimized [1]. Advanced AI models, trained on extensive code and language data, can generate code, detect bugs, and solve complex problems. These innovations can democratize programming, speed up development, and bridge skill gaps for beginners [2]. Leading this revolution are LLMs, such as ChatGPT, DeepSeek, and Gemini, each equipped with sophisticated natural language processing and code generation capabilities [3]. While these models excel in automating coding tasks, they vary in architecture, training, and performance. Despite their availability, systematic comparisons on effectiveness, reliability, and real-world applicability are lacking. This study fills this gap by analyzing ChatGPT, DeepSeek, and Gemini in Python code generation for various algorithmic challenges.

The advent of AI-based code generation tools has both enthusiasm and skepticism within the software development community [4]. Proponents argue that these tools can ac-

celerate development cycles, minimize repetitive tasks, and empower developers to focus on high-level design and creative problem-solving. Conversely, critics caution that AI-generated code may be less robust, secure, and maintainable, especially in mission-critical applications [5]. A core challenge lies in understanding how different models perform under varying conditions, including problem complexity, input ambiguity, and computational constraints [6]. For example, one model might produce syntactically correct code yet lack algorithmic efficiency or overlook critical edge cases, while another might generate compact code at the expense of readability and scalability. These trade-offs highlight the need for a systematic framework to rigorously assess AI code generation tools beyond anecdotes and isolated cases.

This study establishes a systematic framework for benchmarking AI-based code generation tools, analyzing state-of-the-art models like ChatGPT, DeepSeek, and Gemini in terms of code quality, functional robustness, and computational efficiency. Using a curated dataset of algorithmic challenges from Codeforces [7], it evaluates the performance of the models in multiple difficulty levels and core programming paradigms through a structured protocol focusing on accuracy, code quality and efficiency. By integrating these criteria, the study advances a standardized methodology for assessing AI-generated code while offering actionable insights for software development and AI research. Unlike prior studies that assess models in isolation, this work provides a unified, data-driven comparison, identifying persistent challenges such as sensitivity to underspecified requirements and suboptimal resource allocation. The findings guide both researchers in refining AI coding assistants and practitioners in selecting optimal tools based on project needs, ensuring a balance between development speed, computational efficiency, and maintainability.

Organization: The rest of the paper is organized as follows: Section II reviews related work and existing benchmarks; Section III defines the problem; Section IV outlines the methodology and analysis framework; Section V discusses the results; and Section VI concludes the paper.

II. RELATED WORK

In 2023, K. Shahzad et al. exploited LLMs to propose a framework for automated solution development in smart grids [8]. While the proposed framework was modular and scalable,

it did not include benchmarking to compare the LLMs used for automated code generation. Furthermore, it lacked a discussion on the advantages and disadvantages of using one model over another for developing automated software solutions for smart grids. In the same year, C. Qian et al. harnessed the power of large language models through communicative agents to propose a novel framework for automated software development [9]. This framework systematically decomposes the software development lifecycle by breaking tasks into sequential sub-tasks, enabling a more focused approach to achieving the desired outcomes. While it effectively generates software with minimal bugs, its primary focus is on simpler tasks, lacking analysis of more complex ones. Additionally, the study does not provide benchmarking details to assess the performance of different LLMs across various coding challenges.

In the same year 2023, J. Liu et al. introduced EvalPlus, a framework for evaluating the functional correctness of LLM-generated code by expanding test cases through LLM and mutation-based strategies [10]. While their approach improves code synthesis benchmarking, it focuses solely on synthetic evaluation, overlooking other benchmarking aspects and failing to utilize existing datasets with automated testing. In 2024, Yue Liu et al. conducted a systematic study on the quality of ChatGPT-generated code, analyzing 4,066 programs across Java and Python [11]. While they explored maintainability and self-repairing capabilities, their work focused solely on ChatGPT without comparing it to other industry models. Additionally, they did not provide benchmarking for correctness and efficiency or leverage existing tools for problem selection and automated testing. In 2025, M. Manik compared ChatGPT and DeepSeek for Python code generation using online coding challenges [12]. While the study leveraged standard tools, the problem selection was relatively simple, failing to capture complexities that could better assess model capabilities. The benchmarking focused on basic metrics rather than multifaceted aspects, and the methodology lacked systematic formalization for reproducibility. Additionally, the analysis did not include Gemini, limiting the scope of comparison.

III. PROBLEM FORMULATION

The study focuses on defining a systematic comparative analysis of the popular LLMs for code generation, i.e., ChatGPT, DeepSeek, and Gemini, by rigorously evaluating their performance in generating Python code. It involves benchmarking each model on standardized algorithmic challenges based on three key dimensions: correctness (accuracy of first-attempt solutions), efficiency (speed of generating accepted solutions), and code quality (readability and adherence to coding standards). The challenges are set up using a curated dataset of algorithmic problems under controlled conditions, e.g., identical prompts, limited attempts, and fixed temperature settings, to rigorously uncover the inherent trade-offs and performance nuances of each LLM, enabling a clear understanding of how they handle varying levels of problem

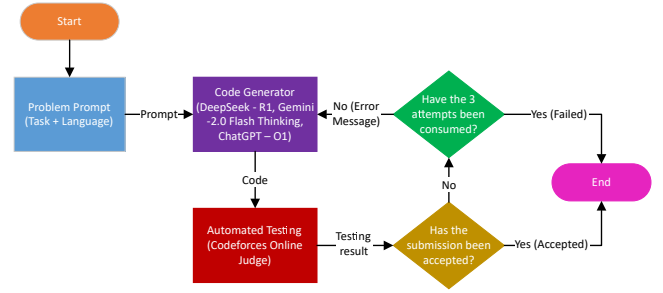


Fig. 1. High-level flowchart of the experimental procedure.

complexity and resource constraints, thereby informing strategic selection based on specific development needs.

IV. METHODOLOGY

The methodology for this study is designed to systematically evaluate the code-generation capabilities of ChatGPT (version O1), DeepSeek (version R1), and Gemini (version 2.0 Flash thinking) across a range of Python programming problems. The evaluation framework focuses on three core dimensions: correctness, efficiency, and code quality, leveraging a combination of standardized coding challenges and quantitative metrics. The experimental framework is designed to be rigorous and reproducible, ensuring that the outcomes not only highlight performance differences but also reveal the underlying strengths and limitations of each model.

Five problems are selected from the Codeforces online judge platform, spanning difficulty ratings from 800 (beginner) to 2000 (advanced). These problems are chosen to represent diverse algorithmic paradigms, including mathematical logic, greedy algorithms, and dynamic programming, ensuring a balanced assessment of the adaptability of each model. For instance, "50A. Domino Piling" tests basic arithmetic logic, while "1370D. Odd-Even Subsequence" requires advanced optimization techniques. Each model is tasked with generating Python code for these problems using identical prompts, formatted to exclude explanations or comments. To maintain consistency, the models are configured with a temperature setting of 0.2 to minimize randomness, and a maximum 3 attempts per problem, with error feedback provided after each failed submission. Initially, we developed a set of manually curated test cases; however, early experiments revealed that these handcrafted cases can be insufficient for exposing nuanced performance differences. Consequently, we transitioned to utilizing dynamic test cases of CodeForces, which provided real-time feedback and detailed error messages, thereby facilitating iterative improvements in the generated code.

As presented in Fig. 1, the experimental procedure is executed in several distinct phases. First, each model is prompted to generate a Python code solution for a given problem. Upon submission of the generated code to the online judge, we record the immediate verdict, noting whether the solution is "Accepted" or require further refinement. In cases where the code is rejected, the error messages provided by the judge are relayed back to the respective model, prompting it to modify

TABLE I
COMPARISON OF RESULTS BETWEEN DEEPSEEK, GEMINI, AND CHATGPT ON CODEFORCES PROBLEMS

Problem (Codeforces)	Complexity	Model	Attempts	Reasoning Time (s)	Runtime (ms)	Memory (KB)	Active Lines	Issues (Pylint)	Code Rating (Pylint)
50A. Domino Piling	800	DeepSeek-R1	1	14	124	40	2	2	0
		Gemini-2.0	3	5	124	12	6	5	1.67
		ChatGPT-O1	3	8	154	20	5	5	0
2037A. Twice	800	DeepSeek-R1	1	10	61	100	9	2	7.78
		Gemini-2.0	1	3	62	8	10	4	6
		ChatGPT-O1	1	9	62	8	13	4	6.92
1690D. Black and White	1000	DeepSeek-R1	1	83	140	728	16	2	8.75
		Gemini-2.0	1	3	109	708	38	5	8.65
		ChatGPT-O1	1	8	93	2920	22	9	6.4
1891C. Smilo and Monsters	1500	DeepSeek-R1	1	281	202	27064	27	10	6.3
		Gemini-2.0	Failed (3 attempts)	N/A	N/A	N/A	N/A	N/A	N/A
		ChatGPT-O1	1	173	281	26100	19	4	7.89
1370D. Odd-Even Subsequence	2000	DeepSeek-R1	2	310	937	26100	50	14	6.96
		Gemini-2.0	2	4	1312	26100	35	9	7.19
		ChatGPT-O1	1	18	1499	26132	40	18	5

its approach. This cycle is repeated for up to three attempts until a correct solution is achieved. The approach allows us to gauge not only the initial performance of each model but also their ability to learn from iterative feedback.

Subsequently, we examine the quality of the generated code using static analysis tools such as Pylint, by integrating it within the VS Code environment. These tools assess the adherence of the code to established coding standards, quantify the number of stylistic issues, and provide an overall code rating. In addition, we evaluate the conciseness of the code by measuring the active lines, excluding comments and blank spaces, thus capturing the efficiency with which each model expressed its solutions. Finally, the study assesses performance through three primary metrics, such as correctness, efficiency, and code quality. For correctness, we propose a straightforward ratio where the total number of accepted attempts across all problems is divided by the total number of attempts made. This is expressed as:

$$Correctness = \frac{\sum Accepted\ Attempts}{\sum Total\ Attempts} \times 100 \quad (1)$$

Efficiency is calculated as the ratio of accepted attempts to the total reasoning time, as follows:

$$Efficiency = \frac{f}{\bar{T}} \quad (2)$$

Here, $f = \frac{N_{first}}{N_{accepted}}$ represents the ratio of first-attempt accepted solutions to total accepted solutions, $\bar{T} = \frac{1}{N_{accepted}} \sum_{i=1}^{N_{accepted}} T_i$ is the average reasoning time per accepted solution.

Code quality, which encompasses readability and maintainability, is evaluated by considering the number of issues identified in the generated code relative to the volume of the code produced. One approach to quantify this is to compute

the inverse of the ratio of the total number of issues to the total number of active lines of code [12], as follows:

$$Code\ Quality = \frac{1}{\left(\frac{\sum Total\ Issues}{\sum Active\ Lines\ of\ Code} \right)} \quad (3)$$

V. RESULTS AND DISCUSSION

The experimental findings from our comparative analysis of ChatGPT, DeepSeek, and Gemini reveal a nuanced landscape in which each model exhibits distinct performance profiles under varied conditions. As presented in Table I, DeepSeek consistently demonstrated high correctness, solving 83% of the evaluated problems (Table II) and often achieving solutions on the first attempt. This reliability was particularly evident in problems such as "50A. Domino Piling" and "2037A. Twice" where DeepSeek delivered optimal runtimes of 124 ms and 61 ms, respectively, with minimal code complexity, ranging from 2 to 9 active lines. However, as problem complexity increased, particularly in "1891C. Smilo and Monsters" and "1370D. Odd-Even Subsequence", the reasoning time for providing the

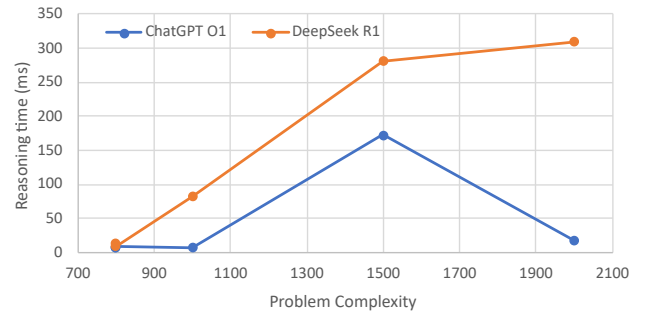


Fig. 2. Comparison between ChatGPT-O1 and DeepSeek-R1 in terms of reasoning time with respect to problem complexity.

TABLE II
COMPARISON OF DEEPSEEK, GEMINI, AND CHATGPT BASED ON
CORRECTNESS, EFFICIENCY, AND CODE QUALITY

Model	Correctness	Efficiency	Code Quality
DeepSeek-R1	83.33	0.006	3.47
Gemini-2.0	40.00	0.133	3.87
Flash thinking			
ChatGPT-O1	71.43	0.019	2.48

solution escalated to 281 and 310 seconds, respectively. This trend suggests a trade-off between accuracy and computational efficiency, where complex tasks require significantly more processing time, as indicated in Fig. 2.

Gemini exhibited contrasting characteristics, with reasoning times consistently under 5 seconds across all solvable problems (Table I), making it 3 to 4 times faster than the other models. While this rapid performance enabled a 62 ms runtime for "2037A. Twice" (comparable to solution provided by DeepSeek), the 40% overall correctness rate of Gemini (Table II) revealed critical limitations. Most notably, it failed entirely on the 1500-difficulty problem "1891C. Smilo and Monsters" despite three attempts, indicating challenges with multi-step reasoning. This pattern suggests that the speed optimization of Gemini comes at the cost of solution robustness, particularly for higher-complexity problems requiring backtracking or advanced algorithmic strategies.

ChatGPT demonstrated intermediate capabilities, balancing 71.4% correctness with superior code quality metrics. As evidenced by Pylint ratings in Table I, ChatGPT solutions averaged 2.48 code quality versus 3.47 of DeepSeek and 3.87 of Gemini, despite requiring multiple iterations for some problems. For instance, in "50A. Domino Piling", ChatGPT needed three attempts but ultimately produced solution with zero code rating and moderate memory usage (20KB). However, this occasionally coincided with suboptimal resource utilization, e.g., its solution for "1690D. Black and White" consumed 2920 KB memory compared to 728 KB by DeepSeek, highlighting context-dependent trade-offs between code elegance and computational efficiency.

As summarized in Table II, no model dominated across all dimensions. The high correctness and reliability of DeepSeek make it ideal for scenarios where accuracy and robustness are paramount, despite higher reasoning times. The sub-second reasoning of Gemini suggests utility for rapid prototyping in low-stakes scenarios, provided solutions undergo rigorous validation. The balance of ChatGPT with 71.4% correctness and maintainable code positions it as a versatile tool for iterative development processes. These findings underscore the need for context-aware model selection based on task-specific goals like correctness, efficiency, and code quality, rather than assuming one model fits all. Although the study offers a systematic framework for evaluating AI-generated code, its limitations include a focus on Python-specific evaluation, reliance on algorithmic problems rather than real-world software engineering tasks, and exclusion of Codeforces problems exceeding 2000 complexity.

VI. CONCLUSION

This study conducted a systematic evaluation of ChatGPT, DeepSeek, and Gemini for automated Python code generation, highlighting the intrinsic trade-offs among correctness, efficiency, and code quality. Our findings revealed that DeepSeek excels in correctness, consistently solving a high percentage of problems with reliable performance, albeit with increased reasoning times on complex tasks. In contrast, the reasoning time of Gemini for delivering solutions within seconds, positions it as an attractive option for rapid prototyping, though its lower correctness rate on higher-difficulty challenges limits its robustness. ChatGPT emerges as a balanced option, combining moderate correctness with maintainable code quality, yet it sometimes requires additional iterations and exhibits occasional inefficiencies. Overall, the comparative analysis underscores the importance of aligning the choice of AI code generation tool with the specific requirements of the task at hand. Future research aims to develop hybrid approaches that leverage the strengths of existing models while addressing complex challenges and refining benchmarking metrics.

REFERENCES

- [1] I. Udoidiok, H. Reza, and J. Zhang, "Exploring ai integration in software development: Case studies and insights," in *NAECON 2024-IEEE National Aerospace and Electronics Conference*. IEEE, 2024, pp. 375–380.
- [2] A. Rajbhoj, A. Somase, P. Kulkarni, and V. Kulkarni, "Accelerating software development using generative ai: Chatgpt case study," in *Proceedings of the 17th innovations in software engineering conference*, 2024, pp. 1–11.
- [3] W. Hou and Z. Ji, "Comparing large language models and human programmers for generating programming code," *Advanced Science*, vol. 12, no. 8, p. 2412279, 2025.
- [4] A. Sergeyuk, Y. Golubev, T. Bryksin, and I. Ahmed, "Using ai-based coding assistants in practice: State of affairs, perceptions, and ways forward," *Information and Software Technology*, vol. 178, p. 107610, 2025.
- [5] R. Wang, R. Cheng, D. Ford, and T. Zimmermann, "Investigating and designing for trust in ai-powered code generation tools," in *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*, 2024, pp. 1475–1493.
- [6] T. Coignon, C. Quinton, and R. Rouvoy, "A performance study of llm-generated code on leetcode," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, 2024, pp. 79–89.
- [7] A. Y. Ramahalim, C. E. Karuniawan, E. S. Purwanto, and M. K. Ario, "Trend analysis in competitive programming using topic modeling," *Procedia Computer Science*, vol. 245, pp. 591–600, 2024.
- [8] K. Shahzad, S. Iqbal, and M. M. Fraz, "Automated solution development for smart grids: Tapping the power of large language models," in *2023 17th International Conference on Engineering of Modern Electric Systems (EMES)*. IEEE, 2023, pp. 1–4.
- [9] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong *et al.*, "Chatdev: Communicative agents for software development," *arXiv preprint arXiv:2307.07924*, 2023.
- [10] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," *Advances in Neural Information Processing Systems*, vol. 36, pp. 21 558–21 572, 2023.
- [11] Y. Liu, T. Le-Cong, R. Widyasari, C. Tantithamthavorn, L. Li, X.-B. D. Le, and D. Lo, "Refining chatgpt-generated code: Characterizing and mitigating code quality issues," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 5, pp. 1–26, 2024.
- [12] M. M. H. Manik, "Chatgpt vs. deepseek: A comparative study on ai-based code generation," *arXiv preprint arXiv:2502.18467*, 2025.